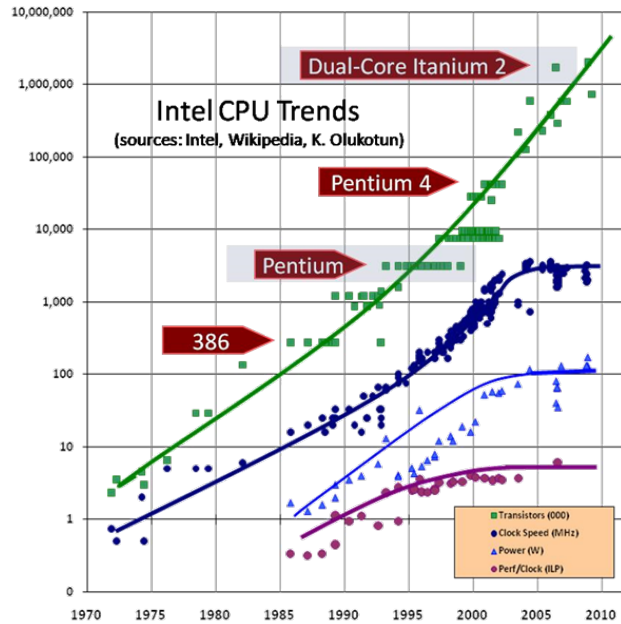


1 parallel STL

Obliczenia równoległe na wielu rdzeniach

Dopóki nie stworzymy komputerów kwantowych będziemy musieć zaakceptować ograniczenia świata fizycznego i procesorów bazujących na idei tranzystora krzemowego:



Rysunek 1: Parallel STL

Nie wszystkie aplikacje (a dokładniej ważne operacje aplikacji) mogą być przyspieszone przez równoległe rozdzielenie na wiele procesów. To prawda, że niektóre problemy, takie jak kompilacja, są prawie idealnie równoległe. Ale inne nie są; zwykły kontrprzykład jest taki, że fakt, że wyprodukowanie dziecka zajmuje kobiecie dziewięć miesięcy, nie oznacza, że dziewięć kobiet może wyprodukować jedno dziecko w ciągu jednego miesiąca. Prawdopodobnie wcześniej spotkałeś się z tą analogią. Ale czy zauważyłeś problem z w tej analogii? Oto podchwytliwe pytanie, które można zadać osobie, która jej używa: Czy możesz z tego wyciągnąć wniosek, że Problem Ludzkiego Dziecka nie jest z natury podatny na równoległość? Zazwyczaj ludzie odnoszący się do tej analogii myślą się szybko, stwierdzając, że demonstruje ona problem z natury nierównoległy, ale w rzeczywistości niekoniecznie jest poprawny. Jest to z natury rzeczy nierównoległy problem, jeśli celem jest wyprodukowanie jednego dziecka. Jest to właściwie idealnie równoległy problem, jeśli celem jest wyprodukowanie wielu dzieci! Znajomość prawdziwych celów może mieć ogromne znaczenie. Ta podstawowa zasada zorientowana na cel to coś, o czym należy pamiętać, rozważając,

czy i jak zrównoleglić oprogramowanie.

Na razie zatrzymujemy się w zakresie 3GHz. W związku z tym programy jednowątkowe prawdopodobnie nie będą już teraz dużo szybsze, z wyjątkiem korzyści wynikających z dalszego wzrostu rozmiaru pamięci podręcznej typu cache.

Wydażność i optymalizacja wydażności zyskają na znaczeniu, a nie tracą.

Języki programowania i systemy będą coraz częściej zmuszane do radzenia sobie ze współbieżnością.

C++ 17 jest krokiem w tym kierunku i pozwala nam korzystać z większej mocy obliczeniowej: odblokowuje funkcję automatycznej wektoryzacji / automatycznej równoległości dla algorytmów w bibliotece standardowej STL.

Pomysł jest dość prosty. Standardowy szablon (STL) zawiera ponad 100 algorytmów do wyszukiwania, liczenia i manipulowania zakresami i ich elementami. Z C++ 17, 69 z nich jest przeciążonych i dodano kilka nowych. Przeciążony i nowy algorytm można wywołać za pomocą tak zwanej polityki wykonywania (execution policy). Korzystając z polityki wykonywania, można określić, czy algorytm powinien działać sekwencyjnie, równolegle, czy równolegle i wektorowo:

```
std::algorithm_name(policy, /* normal args... */)
```

przykład:

```
std::vector<int> v = genLargeVector();  
// sortuj wektor  
std::sort(std::execution::par, v.begin(), v.end());
```

Opisy tych polityk wykonywania algorytmów znajdują się tutaj.

Osobiście w dokumentacji najbardziej podoba mi się komentarz, że “Additional execution policies may be provided by a standard library implementation (possible future additions may include std::parallel::cuda and std::parallel::opencl)”.

2 Zestaw zadań

2.1 Zadanie 1

Opisz wykorzystując funkcjonalne przykłady występujące w C++17 execution policy:

```
std::execution::sequenced_policy,
std::execution::parallel_policy,
std::execution::parallel_unsequenced_policy,
```

2.2 Zadanie 2

Wykonanie bardzo prostego programu:

```
#include <vector>
#include <algorithm>
#include <execution>
#include <mutex>

int main() {
    std::vector<int> vec(10'000'000);
    std::iota(vec.begin(), vec.end(), 0);
    std::vector<int> output;

    std::for_each(std::execution::par, vec.begin(), vec.end(),
        [&output](int& elem) {
            if (elem % 2 == 0) {
                output.push_back(elem);
            }
        });
}
```

zakończy się tragicznie z punktu widzenia programisty (core dumped). Wykorzystując `std::lock_guard` zmodyfikuj lambda expression tak aby nie dochodziło do niesynchronizowanego dostępu do wektora wyjściowego. Porównaj wydajność przy wykorzystaniu polityki typu seq.

2.3 Zadanie 3

Wybierz z listy algorytmów:

adjacent_difference	inplace_merge	replace_copy
adjacent_find	is_heap	replace_copy_if
all_of	is_heap_until	replace_if
any_of	is_partitioned	reverse
copy	is_sorted	reverse_copy
copy_if	is_sorted	rotate
copy_n	is_sorted_until	rotate_copy
count	lexicographical_compare	search
count_if	max_element	search_n
equal	merge	set_difference
exclusive_scan	min_element	set_intersection
fill	minmax_element	set_symmetric_difference
fill_n	mismatch	set_union
find	move	sort
find_end	none_of	stable_partition
find_first_of	nth_element	stable_sort
find_if	partial_sort	swap_ranges
find_if_not	partial_sort_copy	transform
for_each	partition	transform_exclusive_scan
for_each_n	partition_copy	transform_inclusive_scan
generate	remove	transform_reduce
generate_n	remove_copy	uninitialized_copy
includes	remove_copy_if	uninitialized_copy_n
inclusive_scan	remove_if	uninitialized_fill
inner_product	replace	uninitialized_fill_n
	unique	unique_copy

pięć różnych algorytmów i przedstaw przykład wykorzystania, w którym równoległe obliczenia przyspieszą obliczenia proporcjonalnie do ilości występujących w procesorze rdzeni.

2.4 Zadanie 4

Niektóre z tych algorytmów są nowe:

```
std::for_each_n
std::exclusive_scan
std::inclusive_scan
std::transform_exclusive_scan
std::transform_inclusive_scan
std::parallel::reduce
std::parallel::transform_reduce
```

Ósmy algorytm `std::for_each` różni się i też uważamy go za nowy, ponieważ oprócz dodatkowej polityki wykonywania teraz zwracanym typem jest `void`. Przygotuj na podstawie dokumentacji przykłady wykorzystania tych algorytmów. W przypadku `reduce` wykorzystaj alfa expression

```
[(auto fir, auto sec){ return fir + ":" + sec; }]
```

do zdefiniowania BinaryOp w celu dodawania stringów.